

Precision-first long-read error correction with support-conditioned local edit evidence

Shane Jayasundera and Eesha Kurella

Department of Computer Science, University of Maryland | CMSC701 Final Project Report

Abstract

Long-read sequencing enables assembly, phasing, and structural-variant discovery across repetitive regions, but Oxford Nanopore reads remain error-prone enough that read-level correction can alter downstream analyses. The central challenge is not simply maximizing edits: in diploid and repeat-rich regions, aggressive correction can erase true haplotypes or repeat-copy differences. We developed Omega Safe SeqEdit, a compact research codebase for conservative long-read correction as support-conditioned edit prediction. Starting from an interim transformer sequence-to-sequence prototype, we rebuilt the system around explicit COPY/SUB/INS/DEL decisions, deletion-aware labels, support-rule baselines, hybrid decoding, real-data abstention, and candidate-level forensic diagnostics. Synthetic benchmarks confirm that the pipeline can learn designed substitutions, insertions, and deletions while avoiding false edits. HG002 chr20 windows expose the harder real-data bottleneck: support-majority and neural-only correction increase identity but introduce many false hard edits. The final strict hybrid policy avoids false edits but currently collapses to no-edit on HG002, motivating a shift from per-site pileup scoring to local-window, whole-read evidence and haplotype-aware abstention.

Keywords: long-read sequencing; Oxford Nanopore; error correction; overcorrection; support reads; haplotype-aware decoding

Introduction

Long-read sequencing observes long-range structure, repeats, and haplotype phase directly in individual reads. Those properties are central to human genome assembly and structural-variant analysis, but the reads still contain substitution, insertion, and deletion errors. Error correction attempts to transform a noisy target read into a more accurate molecule-level sequence using overlapping support reads and, for training or evaluation, truth-aligned data.

The interim project framed the task as sequence-to-sequence translation from noisy DNA to corrected DNA. That system established an end-to-end ONT pipeline with target, support, and truth windows; an edit-token transformer; and baselines. It also revealed the key failure mode: naively conditioning on overlap/support reads can be worse than a target-only model, because support evidence is not uniformly trustworthy.

The final project therefore reframes correction as conservative support-conditioned edit prediction. The method should copy by default, apply hard edits only when evidence is safe, and evaluate overcorrection explicitly. A missed edit leaves an existing sequencing error; a false edit can overwrite a real variant or haplotype. This asymmetry is the reason identity alone is not a sufficient objective.

The biological motivation is strongest in exactly the regions that make correction difficult. Heterozygous variants, tandem repeats, homopolymers, and local indels create settings where two locally plausible sequences may both be supported by reads. A method that blindly follows majority support can convert a read from one haplotype into another, collapse repeat-copy differences, or propagate an alignment artifact. For downstream assembly, phasing, and variant calling, such overcorrection can be more damaging than retaining a small number of original read errors.

Recent systems motivate this caution. DeepConsensus showed the value of gap-aware alignment modeling; HERRO and DeChat emphasize haplotype- and repeat-aware correction; and hifieval evaluates correct corrections, undercorrections, and overcorrections rather than identity alone. Our central question is: can a small, editable, support-conditioned and deletion-aware model beat conservative consensus on small long-read benchmarks without increasing overcorrection?

The practical goal is intentionally modest. Rather than building a large production system, we want the smallest architecture that can answer whether support-conditioned editing can be safe. This led to a codebase organized around tiny modules, JSONL debug examples, fast synthetic tests, Mac-runnable real-data presets, and notebooks that orchestrate scripts rather than hiding core logic. The resulting implementation is as much an experimental instrument as it is a model.

Methods

Data and benchmark design

The codebase is organized around three benchmark levels. The smallest synthetic debug set contains designed copy, substitution, insertion, deletion, homopolymer, and repeat-ambiguity cases. A curated noisy synthetic slice adds shallow support, false-support examples, neighboring edits, homopolymer deletions, and cases where support-rule correction is wrong. The real benchmark uses HG002 chromosome 20 windows with aligned ONT reads and reference/truth annotations when available.

Every example is stored as inspectable JSONL: target sequence, support-aligned sequences, truth sequence, edit labels, delete-candidate and delete-length labels, and explicit features such as base counts, insertion/deletion support, agreement, entropy, homopolymer length, repeat flags, and optional variant/confident-region masks. This format made small-footprint debugging more important than compression or cluster-scale throughput.

For real data, truth is constructed from HG002 reference-aligned regions, high-confidence intervals, and variant annotations where available. This does not make truth perfect: an apparent false edit can reflect phasing, reference bias, low-confidence truth, or local alignment ambiguity. For that reason, the evaluation reports both sequence-level scores and candidate-level audit tables rather than treating one aggregate identity number as definitive.

The required baselines are deliberately simple and honest. The no-edit baseline outputs the target read unchanged. Conservative consensus edits a target base only when support agreement is strong. A support-rule baseline makes explicit COPY/SUB/INS/DEL decisions from pileup evidence. Learned variants include target-only, full neural, and full hybrid decoding. External SOTA tools are treated as future comparison hooks until the method can first beat no-edit and consensus safely on local HG002 windows.

Architecture

Omega Safe SeqEdit predicts local edit actions aligned to target-read positions rather than generating an unconstrained corrected sequence. The target path uses DNA token embeddings and a small transformer encoder. The support path uses per-position pileup and rule features rather than raw multi-read attention. Factorized heads predict edit type, substitution payload, insertion payload, deletion length, and allow/confidence diagnostics.

This design deliberately separates the questions 'should we edit?', 'what family of edit is it?', and 'what payload or length should be used?'. That separation made it possible to discover failures such as insertion payload errors, DEL as a generic fallback, support-rule false positives, and neural-only no-edit or over-edit collapse.

Training also follows the precision-first framing. The loss includes edit-type cross entropy, payload losses on gold hard-edit positions, deletion-specific objectives, margins against COPY at true hard edits, and penalties for unsupported hard edits. Early debugging used overfit protocols for single SUB, INS, DEL, and mixed examples before allowing benchmark claims. This staged process prevented threshold tuning from hiding failures in the underlying edit heads.

Conservative decoding

The decoder is precision-first. Rule-negative positions veto neural hard edits by default. Support-rule substitutions are treated as candidates rather than forced edits on HG002. Insertions and deletions require strong evidence, non-repeat context, and allow-gate approval. Candidate allowlists include SHA-256 hashes and true/false counts that are asserted against evaluated summaries to prevent stale-output errors.

The newest evidence module moves beyond per-site scoring. For each support-rule substitution candidate it exports target and candidate-corrected windows, support-read snippets, COPY-versus-SUB local-window scores, counts of reads better explained by each hypothesis, ambiguous-read counts, cluster margin, flank consistency, strand balance, mapping-quality summaries, nearby indel and mismatch density, repeat flags, and variant-context features.

Evaluation

We report identity and normalized edit distance, but the primary safety metrics are false hard edits, overcorrection, corrected edits, and usable score: $\text{usable} = \text{identity} - 0.5 * \text{overcorrection} - 0.5 * \text{hard-edit false-positive rate}$. This score intentionally penalizes unsafe edit activity even when identity appears to improve.

Diagnostic outputs include confusion among COPY, SUB, INS, and DEL; per-base substitution and insertion recall; gate and veto reasons; false-edit dumps; vetoed-true-edit dumps; and ranked-candidate allowlist checks. These diagnostics became central because the most common failures were structural rather than stochastic: SUB candidates that looked locally obvious, insertions with wrong payload anchoring, and deletions near homopolymers or neighboring edits.

Results

Table 1. Synthetic benchmark outcomes.

Benchmark	Method	Usable	Corrected	False
mac_debug	no_edit	0.9897	0	0
mac_debug	consensus	0.9966	0.667/ex	0
mac_debug	support_rule	0.9996	0.958/ex	0
mac_debug	full_hybrid	0.9987	0.875/ex	0
noisy small	no_edit	0.9922	0	0
noisy small	support_rule	0.9121	n/a	5
noisy small	full_hybrid	0.9970	11 total	0

The clean synthetic benchmark confirms that all edit machinery works end to end. The support-rule baseline is strongest because the synthetic support is designed to be informative. The hybrid model corrects most designed hard edits without false positives. The curated noisy slice is more revealing: raw support rules introduce false edits, whereas conservative hybrid decoding preserves zero false edits while still correcting 11 true edits.

This synthetic success was necessary but not sufficient. It proved that label generation, factorized heads, conservative decode, and benchmark exports were functioning. It did not prove real-world safety, because synthetic reads do not reproduce all sources of support ambiguity: mixed haplotypes, mapping artifacts, repetitive paralogs, VCF/truth boundary effects, and local alignment shifts around indels.

HG002 chr20

Table 2. HG002 12x Mac-runnable test summaries.

Method	Usable	Identity	Corrected	False
no_edit	0.9663	0.9663	0	0
consensus	0.7773	0.9682	960	1278
support_rule	0.7282	0.9678	1001	1528
full_neural	0.5447	0.9641	959	6255
strict full_hybrid	0.9663	0.9663	0	0
audited ranked SUB top-1	0.9638	0.9663	0	1

The HG002 result changes the interpretation. Consensus and support-rule correction improve raw identity but create many false hard edits, causing large usable-score losses. Neural-only decoding is worse because the model applies unsafe edit priors when not protected by the rule-negative veto. Strict hybrid decoding is safe but makes no corrections, matching no-edit. This is the central real-data bottleneck: the system can find plausible edits, but it cannot yet reliably identify which are biologically safe.

Compared with the old solution, the combined system is safer and far more diagnosable, but it is not yet stronger on real correction utility. The old model often collapsed to no-edit or produced unsafe sequence-level changes without enough edit-type visibility. The new system exposes exactly where each decision came from - support rule, neural type head, payload head, allow gate, or veto - but the best verified HG002 policy still equals no-edit. This is progress in scientific control rather than final performance.

A ranked substitution recovery experiment originally appeared to recover one safe edit. After adding allowlist hashing and count assertions, the audited run showed that the top-ranked candidate was actually false. This invalidated the optimistic claim and showed why reproducibility checks are part of the method, not bookkeeping. The current best valid HG002 policy is therefore strict no-recovery, and any positive recovery must beat it under verified allowlists.

The candidate-level audits are now the most valuable result artifact. They show that support-rule true positives and false positives can be nearly indistinguishable under simple scalar evidence. In particular, high support fraction, high margin, and zero entropy are not sufficient for safety on HG002. That observation rules out a broad class of easy threshold fixes and motivates local-window hypothesis scoring.

Failure analysis

False HG002 substitutions often looked strong under scalar features: high support fraction, low entropy, high margin, and high payload confidence. Such evidence is not enough. A pileup column can be confident because reads are shifted by nearby indels, mixed across haplotypes, drawn from repeat copies, or misaligned. Many attempted fixes - larger transformers, lower thresholds, global recovery rules, and pairwise rankers over the same features - increased edit activity without making it safer.

This failure supports a shift in evidence granularity. Instead of asking whether one pileup column favors a different base, the next decoder should ask whether a local corrected sequence explains whole support reads better than COPY, while respecting strand balance, mapping quality, repeat context, nearby variants, and local indel ambiguity. This is the same biological caution that motivates haplotype- and repeat-aware modern correction systems.

The immediate implementation path is therefore not a larger transformer. It is a better evidence unit: for each candidate correction, cluster support reads into target-like, candidate-like, and ambiguous groups; compare COPY and edited local windows; penalize nearby competing edits; and abstain when evidence is shallow, strand-biased, repetitive, or variant-like. Only after that layer separates safe and unsafe candidates should model capacity or external SOTA-scale benchmarking become the priority.

Discussion

The project produced both a usable framework and an informative negative result. On controlled synthetic tasks, the compact edit-prediction architecture learns substitutions, insertions, and deletions and can be decoded conservatively. On real HG002 windows, however, the present support representation is too local. Real support reads are not independent votes for a single reference-free truth; they may encode haplotypes, repeats, local alignment shifts, or errors in the support itself.

This explains why identity is misleading in this phase. Consensus appears attractive by identity, but its false hard edits make usable score much worse than no-edit. A learned model that simply becomes more willing to edit has the same problem at larger scale. The research target is not maximum edit recall; it is safe true-edit recovery under a near-zero false-positive constraint.

The most promising next step is local-window reranking. For a candidate substitution, generate COPY, candidate-SUB, nearby-indel, and minimal-support-rule hypotheses; score each hypothesis against support reads over a surrounding window; penalize extra edits, haplotype mixing, repeats, and unsupported neighboring changes; and allow the candidate only if the corrected hypothesis wins by a large margin. This converts correction from per-position classification into local hypothesis testing.

The novelty of the current solution is not that it introduces a new large neural architecture. Its contribution is the safety-centered research loop: factorized edit heads, deletion-aware labels, support-rule teachers, hybrid veto/rescue decoding, usable-score model selection, and candidate forensics that explicitly distinguish correction from overcorrection. This makes negative results actionable instead of burying them inside a single identity score.

The current study has limitations. The HG002 benchmark is intentionally small and Mac-runnable, not a final SOTA benchmark. External tools such as HERRO and DeChat were not run end-to-end on the same data. Ground truth depends on reference, VCF, and confident-region assumptions. Candidate evidence currently uses aligned snippets rather than full haplotype-aware realignment. These limitations are precisely why the report avoids claiming state-of-the-art performance.

Conclusion

Omega Safe SeqEdit advances the project from a naive transformer sequence-to-sequence correction model to a precision-first, support-conditioned edit-correction framework. It shows that the pipeline can learn designed edits and that conservative hybrid decoding can suppress synthetic false positives. It also shows that real human ONT correction cannot be solved safely by isolated support-majority pileup features. The next methodological jump is to rank local correction hypotheses using whole-read support, haplotype/variant context, and repeat-aware abstention.

A successful next milestone would be intentionally small: recover 1-5 true HG002 edits with zero verified false edits, then 10-20, then compare against consensus and external tools only after the safe frontier is real on held-out intervals. That progression is more scientifically meaningful than chasing synthetic recall or reporting identity gains that are paid for by overcorrection.

In its current form, the system is not SOTA-competitive on HG002. Its value is that it makes this failure visible, measurable, and actionable. The framework now answers the most important early research question honestly: safe correction is possible on controlled data, but real-data gains require richer evidence before scaling model capacity.

Data and code availability

The code, notebooks, configuration files, and saved summaries are in the local LRS-Error-Correction repository. The primary final implementation is `combined_solution/omega_safe_seqedit`; the clean rebuild is `new_solution/omega_lr_rebuild`; and the original prototype is `old_solution`.

Author contributions

S.J. and E.K. developed the project concept, implementation plan, benchmark framing, and evaluation strategy. S.J. implemented the final codebase, diagnostics, and report synthesis. Both authors contributed to problem formulation, literature framing, interpretation of synthetic and HG002 results, and the safety-first research direction.

Funding and conflicts of interest

This work was completed as a CMSC701 course research project. The authors declare no competing interests. No external funding source influenced the design, analysis, or interpretation of the reported experiments.

Supplementary material

Supplementary artifacts include the final notebook, JSONL example files, saved benchmark summaries, candidate evidence exports, false-edit and vetoed-true-edit audit tables, and the PDF generation script used to render this report.

References

1. Baid G. et al. DeepConsensus improves the accuracy of sequences with a gap-aware sequence transformer. *Nature Biotechnology* 41, 232-238 (2023). <https://www.nature.com/articles/s41587-022-01435-7>
2. Oxford Nanopore Technologies. HERRO: haplotype-aware error correction of ultra-long nanopore reads. <https://nanoporetech.com/resource-centre/herro-haplotype-aware-error-correction-of-ultra-long-nanopore-reads>
3. Liu Q. et al. Repeat and haplotype aware error correction in nanopore sequencing reads with DeChat. *Communications Biology* (2024). <https://www.nature.com/articles/s42003-024-07376-y>
4. hifieval: Evaluation of haplotype-aware long-read error correction. *Bioinformatics* 39, btad631 (2023). <https://academic.oup.com/bioinformatics/article/39/10/btad631/7321114>
5. Salmela L. and Rivals E. LoRDEC: accurate and efficient long read error correction. *Bioinformatics* 30, 3506-3514 (2014). <https://doi.org/10.1093/bioinformatics/btu538>